

UT 10 – BBDD relacionales – Ejercicios

Introducción a estos ejercicios: objetos vs filas

Java es un lenguaje orientado a objetos, así que la forma ideal o más correcta de trabajar con las filas almacenados en una base de datos es no usar directamente los resultsets, sino hacer conversiones a objetos y colecciones de objetos.

Normalmente se hace lo siguiente:

- Se crean clases Java para representar los datos almacenados en las distintas tablas de la BD. En general se usa una clase por tabla, pero puede que en algunos casos se use una clase para aglutinar varias tablas, o que una tabla se divida en varias clases.
- Se crean clases con métodos:
 - Para leer filas de la base de datos: realizan el acceso a la BD, obteniendo un resultset, y devuelven objetos o colecciones de objetos.
 - Para crear filas en la base de datos: reciben un objeto o colección de objetos y crean filas para almacenar los objetos en la base de datos.
 - Para modificar la base de datos: reciben un objeto o colección de objetos y modifican en la base de datos las filas asociadas a los objetos.
 - Para borrar filas de la base de datos: reciben un objeto, una colección de objetos, un identificador de objeto, o una colección de identificadores, y eliminan de la BD las filas asociadas a los objetos.

Estas operaciones se pueden realizar siguiendo patrones de diseño establecidos como el patrón DAO (Data Access Object) o el patrón repositorio.

En los siguientes ejercicios vamos a crear algunas clases que realizarán operaciones de lectura de la base de datos para devolver colecciones, y luego usaremos las colecciones para realizar operaciones con ellas.

También habrá que crear algunas clases para cada una de las tablas que usemos en la base de datos.

Aunque está dividido en varios ejercicios, todos el código que se realice en los ejercicios de esta tanda estará en un unico paquete (elige el nombre que quieras), en el que crearemos sub-paquetes para agrupar el código.

Ejercicio 10 – Creación de clases para las tablas “film”, “actor”, y “film_actor”

Crear un subpaquete “entities”, en el que crearemos las clases para representar las filas de la distintas tablas de la base de datos. Las clases no tienen por qué tener tantos atributos como columnas haya en las tablas.

Crear en este paquete las siguientes clases:

- Actor – Para la tabla “customer” – Columnas utilizadas: actor_id, first_name, last_name.
- Film – Para la tabla “film” – Columnas utilizadas: film_id, title, description, release_year, length, rating
- ActorInFilm – Para la tabla “film_actor” – Columns utilizadas: actor_id, film_id

En estas clases, sobrescribir los métodos equals() y hashCode() para que al comparar objetos de la clase se consideren iguales si tienen el mismo identificador. Esto es necesario para el funcionamiento adecuado de ciertos métodos de colecciones.

Implementar también la interfaz Comparable, para que, siendo consistente con la sobrescritura de equals() y hashCode(), ordenen por defecto por el identificador.

Ejercicio 11 – Creación de clase y métodos para acceder a las tablas

Crear un sub paquete “dao” en el que crearemos las clases para acceder a la base de datos.

Crear las siguientes clases:

- ActorDao: para acceder a los datos de actores.
 - Constructor, que recibe la cadena de conexión, el usuario y la contraseña para conectar en la base de datos.
 - Método getAll(): obtiene todos los actores. Devuelve todos los actores.
 - Método getByld(int id): obtiene el actor por su id. Si no se encuentra el actor devuelve null.
- FilmDao: para acceder a los datos de películas.
 - Constructor, que recibe la cadena de conexión, el usuario y la contraseña para conectar en la base de datos.
 - Método getAll(): obtiene todas las películas. Devuelve todas las películas (interfaz List).
 - Método getByld(int id): obtiene la película por su id. Si no se encuentra la película devuelve null.
- ActorInFilm: para acceder a los datos de la relación entre actores y películas.
 - Constructor, que recibe la cadena de conexión, el usuario y la contraseña para conectar en la base de datos.
 - Método getAll(): obtiene todas las asociaciones ente actores y películas.

Ejercicio 12 – Informe de películas y actores

Crear un programa “FilmActorReport”, en el paquete principal. Detro de este programa, crear un método createFilmActorReport que:

- Recibe tres colecciones:
 - Una lista de películas
 - Una lista de actores
 - Una lista con todas las asociaciones entre actores y películas
- Devuelve un mapa Map<Film, Set<Actor>>, en el que tendremos todas las películas (son la clave del mapa), y para cada una de ellas, todos los actores que participan en ella. Los actores tienen que estar ordenados por nombre y apellidos.

En el programa principal, usar las clases Dao definidas previamente para obtener las tres colecciones, y usar el método createFilmActorReport para, a partir de las tres colecciones obtenidas de la BD, generar el mapa.

Crear y utilizar un método que muestre un pequeño informe (que se mostrará en consola) en el que se muestre en cada línea el título de la película, y a continuación el nombre y apellidos de todos los actores que participan en ella, en orden alfabético por nombre y apellidos.

Ejercicio 13 – Usando Map en lugar de List

Copiar todo el paquete de los ejercicios anteriores, y cambiar las clases Dao para que devuelvan, en el caso de los actores y las películas, en lugar de `List<T>`, `Map<int, T>`. T será Actor en ActorDao y Film en FilmDao. En el mapa, en la clave se almacenará el id del actor / película, y en el valor se almacenará el objeto completo.

Modificar el resto de programas y clases para que se adapten a este nuevo esquema. Observar cómo esto puede hacer que el código de algunos métodos sea mucho más eficiente y fácil de leer.